

PC Build Store Management System

Project Report

Group Members:

Abdul Hanan (Lead)

Haroon Ikram

Aitzaz

Ahmad

Submission Date: 31st May 2026

1. Introduction

We built a PC Build Store Management System using Java Swing. It lets users browse PC builds, check out GPU upgrades, and purchase builds within their budget. Everything is connected - you pick a budget, see what builds fit, buy one, optionally upgrade the GPU, and it shows up on the dashboard.

The project is split into 4 modules, one per person. Each person handles their own GUI, backend, and database stuff. We used Git to collaborate and merge everything together.

Technologies Used

- Java 17
- Swing (GUI)
- MariaDB 10.4 via XAMPP
- MySQL Connector/J 9.7.0
- Apache Ant (build tool)
- NetBeans IDE
- Git + GitHub

2. Problem Statement

There is no easy way for a PC shop to show builds organized by budget, let customers see GPU upgrade options with prices, generate bills when someone buys a build, and keep track of what has been sold. Most existing tools are either too complicated or do not tie everything together. We wanted something simple where all of this works in one place.

3. Objectives

- Build a modular Java Swing app where each person owns their section
- Let users browse and filter PC builds by budget
- Show GPU upgrade options with price and performance info
- Let users upgrade GPU during purchase
- Generate bills and track revenue
- Store everything in MySQL
- Show stats on a dashboard

4. ER Diagram

The database has three tables:

Table	Column	Type	Notes
builds	build_id	INT PK	Auto increment
builds	cpu_model	VARCHAR	e.g. Intel Core i5
builds	cpu_type	VARCHAR	Intel or AMD

builds	gpu_model	VARCHAR	e.g. RTX 4070
builds	gpu_type	VARCHAR	Nvidia or AMD
builds	total_price	INT	50k-500k PKR
builds	performance_score	INT	0-120 scale
gpu_options	gpu_id	INT PK	Auto increment
gpu_options	brand	VARCHAR	Nvidia or AMD
gpu_options	for_budget	INT	Budget tier
gpu_options	price_increase	INT	Cost in PKR
bills	bill_id	INT PK	Auto increment
bills	build_id	INT FK	References builds
bills	final_price	INT	Total cost
bills	purchase_date	DATETIME	Auto timestamp

Relationships

- builds (1) to (many) bills - Each build can be bought multiple times
- gpu_options are independent - no foreign key to builds

5. Module Distribution

Abdul Hanan (Lead) - Dashboard & Reports

- DashboardGUI - main SPA frame with sidebar navigation
- Stat cards showing builds, revenue, avg score, bills
- DBConnection singleton for JDBC
- Schema design and seed data (184 builds + 59 GPUs)
- Fixed purchase filter to exact budget match
- Added GPU options for all budget tiers
- All integration and bug fixes

Haroon Ikram - Build Catalog

- BuildCatalogGUI - table with build listings
- Form for adding/editing builds
- Color coding for CPU and GPU types
- BuildDAO with all CRUD operations
- Fixed brand filter action listener
- Budget filtering and input validation

Aitzaz - GPU Upgrades

- GPUUpgradesGUI - GPU options table
- Brand filter (Nvidia/AMD)
- Form for adding GPU options with labels
- GPUOptionDAO with all CRUD operations

Ahmad - Billing

- BillingGUI - bill history and purchase interface
- Budget selector dropdown (50k-500k)
- Purchase workflow with GPU upgrade option
- After selecting build, user can choose GPU upgrade
- GPU price and score added to final bill
- BillDAO with revenue and bill count tracking

6. Integration Strategy

We used a SPA (Single Page Application) approach. DashboardGUI is the main frame, and clicking sidebar buttons switches between modules using CardLayout.

Data Flow

- Dashboard shows stats from all modules
- Billing connects to BuildCatalog to filter builds by exact budget
- After selecting build, GPU upgrades for that budget are shown
- When you buy a build, it gets deleted from BuildCatalog
- Dashboard stats refresh when you go back to it
- All modules share the same DBConnection singleton

7. Implementation Details

Package Structure

com.project/	
Main.java	- entry point
database/DBConnection	- JDBC connection singleton
models/	- Build, GPUOption, Bill
dao/	- BuildDAO, GPUOptionDAO, BillDAO
ui/	- DashboardGUI, BuildCatalogGUI, GPUUpgradesGUI, BillingGUI

Key Design Decisions

- Singleton for DB connection - reuse one connection
- DAO pattern to separate database logic from UI
- CardLayout for SPA navigation
- Exact budget matching for builds and GPU options

8. Testing

What We Tested

- Database connection works
- All CRUD operations work for builds, GPUs, and bills
- Budget filtering shows only exact budget builds
- GPU upgrade dialog shows for all budget tiers
- Purchase workflow: select build, upgrade GPU, confirm
- Build deleted after purchase
- Dashboard stats update after purchases
- GPU brand filter works correctly
- Form validation catches bad input

Sample Data

- 184 builds (50k-500k PKR range)
- 59 GPU options across all budget tiers
- Bills created through purchase workflow

9. Results

- Dashboard shows real-time stats
- Build Catalog lets you add/edit/delete builds
- GPU Upgrades lets you add GPU options and filter by brand
- Billing lets you buy builds with GPU upgrade option
- All modules are connected and data flows between them
- The app looks good with the dark theme
- Git history shows all our commits and collaboration

10. Conclusion

The project turned out well. We learned how to work as a team using Git, split work into modules that actually integrate, use JDBC to connect Java to MySQL, build a decent-looking Swing GUI, and handle data across multiple tables.

Things That Could Be Better

- Could add user login
- Could add more filtering options
- Could add PDF receipt export
- Could add build comparison feature

Overall, the system does what it is supposed to do and we are happy with it.